

One-dimensional convolutions gave us a way to deal with these inter-token relationships by looking at *windows* of words together. And the pooling layers discussed in chapter 7 were specifically designed to handle slight word order variations. In this chapter, we look at a different approach. And through this approach, you'll take a first step toward the concept of *memory* in a neural network. Instead of thinking about language as a large chunk of data, you can begin to look at it as it's created, token by token, over *time*, in sequence.

8.1 Remembering with recurrent networks

Of course, the words in a document are rarely completely independent of each other; their occurrence influences or is influenced by occurrences of other words in the document:

The stolen car sped into the arena.

The clown car sped into the arena.

Two different emotions may arise in the reader of these two sentences as the reader reaches the end of the sentence. The two sentences are identical in adjective, noun, verb, and prepositional phrase construction. But that adjective swap early in the sentence has a profound effect on what the reader infers is going on.

Can you find a way to model that relationship? A way to understand that “arena” and even “sped” could take on slightly different connotations when an adjective that does not directly modify either occurred earlier in the sentence?

If you can find a way to *remember* what just happened the moment before (specifically what happened at time step t when you're looking at time step $t+1$), you'd be on the way to capturing the patterns that emerge when certain tokens appear in patterns relative to other tokens in a sequence. *Recurrent neural nets* (RNNs) enable neural networks to remember the past words within a sentence.

You can see in figure 8.3 that a single recurrent neuron in the hidden layer adds a recurrent loop to “recycle” the output of the hidden layer at time t . The output at time t is added to the next input at time $t+1$. This new input is processed by the network at time step $t+1$ to create the output for that hidden layer at time $t+1$. That output at $t+1$ is then recycled back into the input again at time step $t+2$, and so on.¹

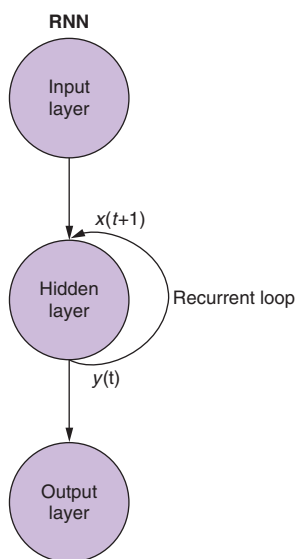


Figure 8.3
Recurrent neural net

¹ In finance, dynamics, and feedback control, this is often called an *auto-regressive moving average* (ARMA) model: https://en.wikipedia.org/wiki/Autoregressive_model.

Although the idea of affecting state across time can be a little mind boggling at first, the basic concept is simple. For each input you feed into a regular feedforward net, you'd like to take the output of the network at time step t and provide it as an additional input, along with the next piece of data being fed into the network at time step $t+1$. You tell the feedforward network what happened before along with what is happening "now."

IMPORTANT In this chapter and the next, we discuss most things in terms of time steps. This isn't the same thing as individual data samples. We're referring to a single data sample split into smaller chunks that represent changes over time. The single data sample will still be a piece of text, say a short movie review or a tweet. As before, you'll tokenize the sentence. But rather than putting those tokens into the network all at once, you'll pass them in one at a time. *This is different than having multiple new document samples.* The tokens are still part of *one* data sample with *one* associated label.

You can think of t as referring to the token sequence index. So $t=0$ is the first token in the document and $t+1$ is the next token in the document. The tokens, in the order they appear in the document, will be the inputs at each *time step* or *token step*. And the tokens don't have to be words. Individual characters work well too. Inputting the tokens one at a time will be *substeps* of feeding the data sample into the network.

Throughout, we reference the current time step as t and the following time step as $t+1$.

You can visualize a recurrent net as shown in figure 8.3: the circles are entire feed-forward network *layers* composed of one or more neurons. The output of the *hidden layer* emerges from the network as normal, but it's also set aside to be passed back in as an input to *itself* along with the normal input from the next time step. This feedback is represented with an arc from the output of a layer back into its own input.

An easier way to see this process—and it's more commonly shown this way—is by *unrolling* the net. Figure 8.4 shows the network stood on its head with two unfoldings of the time variable (t), showing layers for $t+1$ and $t+2$.

Each time step is represented by a column of neurons in the unrolled version of the very same neural network. It's like looking at a screenplay or video frame of the neural net for each sample in time. The network to the right is the *future* version of the network on the left. The output of a hidden layer at one time step (t) is fed back into the hidden layer along with input data for the next time step ($t+1$) to the right. Repeat. This diagram shows two iterations of this unfolding, so three columns of neurons for $t=0$, $t=1$, and $t=2$.

All the vertical paths in this visualization are clones, or views of the same neurons. They are the single network represented on a timeline. This visualization is helpful when talking about how information flows through the network forward and *backward*

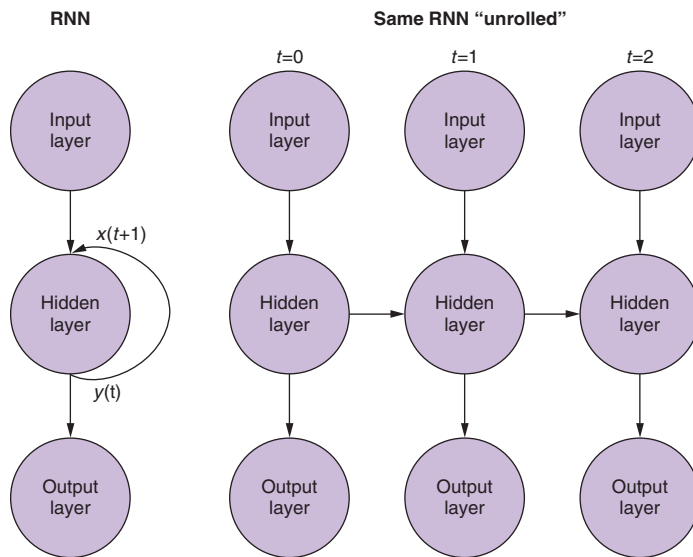


Figure 8.4 Unrolled recurrent neural net

during backpropagation. But when looking at the three unfolded networks, remember that they're all different snapshots of the same network with a single set of weights.

Let's zoom in on the original representation of a recurrent neural network before it was unrolled. Let's expose the input-weight relationships. The individual layers of this recurrent network look like what you see in figures 8.5 and 8.6.

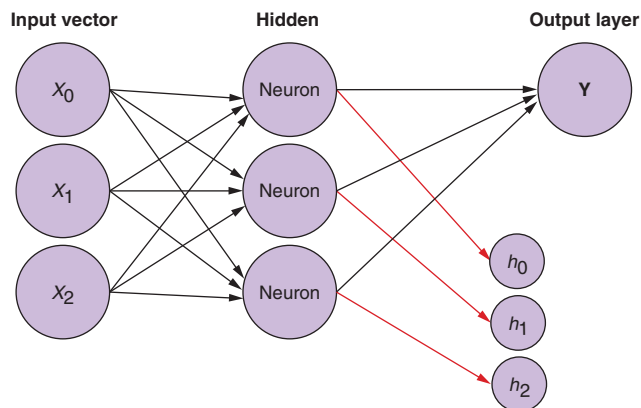


Figure 8.5 Detailed recurrent neural net at time step $t = 0$

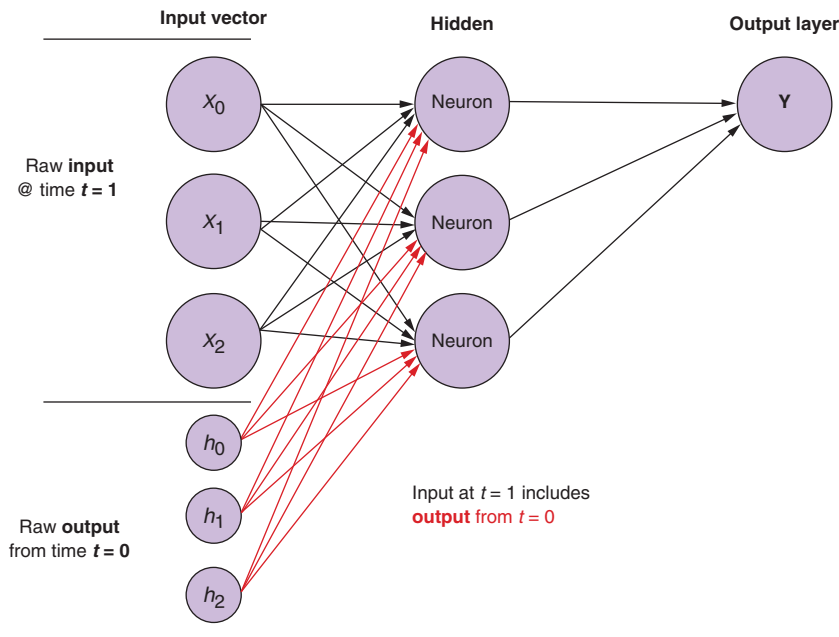


Figure 8.6 Detailed recurrent neural net at time step $t = 1$

Each neuron in the hidden state has a set of weights that it applies to each element of each input vector, as a normal feedforward network. But now you have an additional set of trainable weights that are applied to the output of the hidden neurons from the previous time step. The network can learn how much weight or importance to give the events of the “past” as you input a sequence token by token.

TIP The first input in a sequence has no “past,” so the hidden state at $t=0$ receives an input of 0 from its $t-1$ self. An alternative way of “filling” the initial state value is to first pass related but separate samples into the net one after the other. Each sample’s final output is used for the $t=0$ input of the next sample. You’ll learn how to preserve more of the information in your dataset using alternative “filling” approaches in the section on *statefulness* at the end of this chapter.

Let’s turn back to the data: imagine you have a set of documents, each a labeled example. For each sample, instead of passing the collection of word vectors into a convolutional neural net all at once as in the last chapter (see figure 8.7), you take the sample one token at a time and pass the tokens individually into your RNN (see figure 8.8).

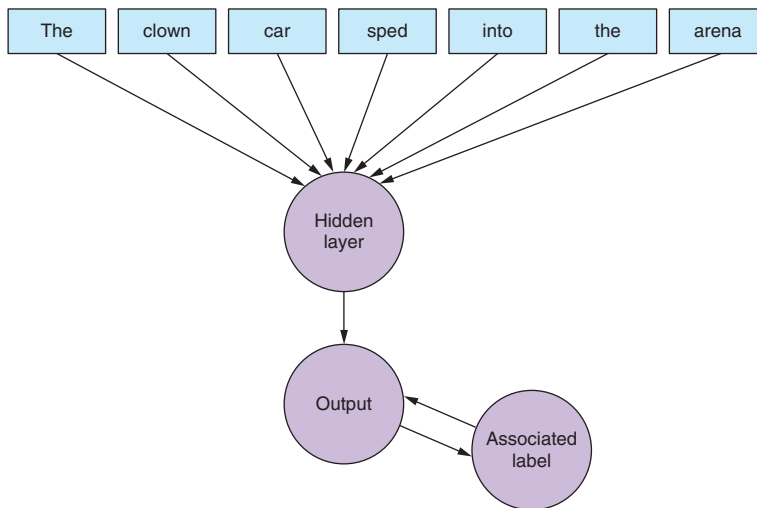


Figure 8.7 Data into convolutional network

In your recurrent neural net, you pass in the word vector for the first token and get the network's output. You then pass in the second token, but you also pass in the output from the first token! And then pass in the third token along with the output from the second token. And so on. The network has a concept of before and after, cause and effect, some vague notion of time (see figure 8.8).

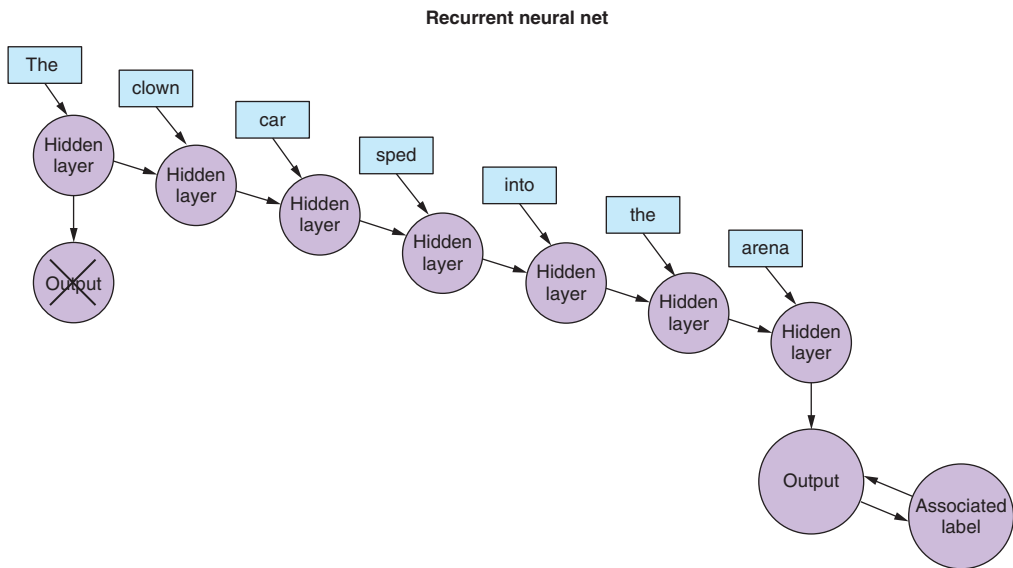


Figure 8.8 Data fed into a recurrent network

Now your network is remembering something! Well, sort of. A few things remain for you to figure out. For one, how does backpropagation even work in a structure like this?

8.1.1 Backpropagation through time

All the networks we’ve talked about so far have a label (the target variable) to aim for, and recurrent networks are no exception. But you don’t have a concept of a label for each token. You have a single label for all the tokens in each sample text. You only have a label for the sample document.

... and that is enough.

Isadora Duncan

TIP We are speaking about tokens as the input to each time step of the network, but recurrent neural nets work identically with any sort of time series data. Your tokens can be anything, discrete or continuous: readings from a weather station, musical notes, characters in a sentence, you name it.

Here, you’ll initially look at the output of the network at the last time step and compare that output to the label. That’ll be (for now) the definition of the *error*. And the error is what your network will ultimately try to minimize. But you now have something that’s a shift from what you had in the earlier chapters. For a given data sample, you break it into smaller pieces that are fed into the network sequentially. But instead of dealing with the output generated by any of these “subsamples” directly, you feed it back into the network.

You’re only concerned with the final output, at least for now. You input each token from the sequence into your network and calculate the loss based on the output from the last time step (token) in the sequence. See figure 8.9.

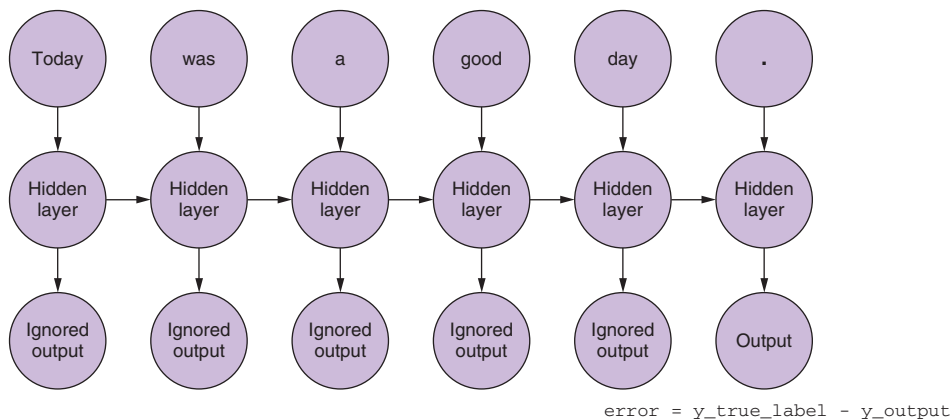


Figure 8.9 Only last output matters here

With an error for a given sample, you need to figure out which weights to update, and by how much. In chapter 5, you learned how to backpropagate the error through a standard network. And you know that the correction to each weight is dependent on how much that weight contributed to the error. You can input each token from your sample sequence and calculate the error based on the output of the network for the previous time step. This is where the idea of applying backpropagation over time seems to mess things up.

Here's one way to think about it: think of the process as time-based. You take a token for each time step, starting with the first token at $t=0$ and you enter it into the hidden neuron in front of you—the next column of figure 8.9. When you do that the network will unroll to reveal the next column in the network, ready for the next token in your sequence. The hidden neurons will unroll themselves, one at a time, like a music box or player piano. But after you get to the end, after all the pieces of the sample are fed in, there will be nothing left to unroll and you'll have the final output label for the target variable in hand. You can use that output to calculate the error and adjust your weights. You've just walked all the way through the computational graph of this *unrolled net*.

At this point, you can consider the whole of the input as static. You can see which neuron fed which input all the way through the graph. And once you know how each neuron fired, you can go back through the chain along the same path and *backpropagate* as you did with the standard feedforward network.

You'll use the chain-rule to backpropagate to the previous layer. But instead of going to the previous layer, you go to the layer in the *past*, as if each unrolled version of the network were different (see figure 8.10). The math is the same.

The error from the last step is backpropagated. For each “older” step, the gradient with respect to the more recent time step is taken. The changes are aggregated and applied to the single set of weights after all the individual tokenwise gradients have been calculated, all the way back to $t=0$ for that sample.

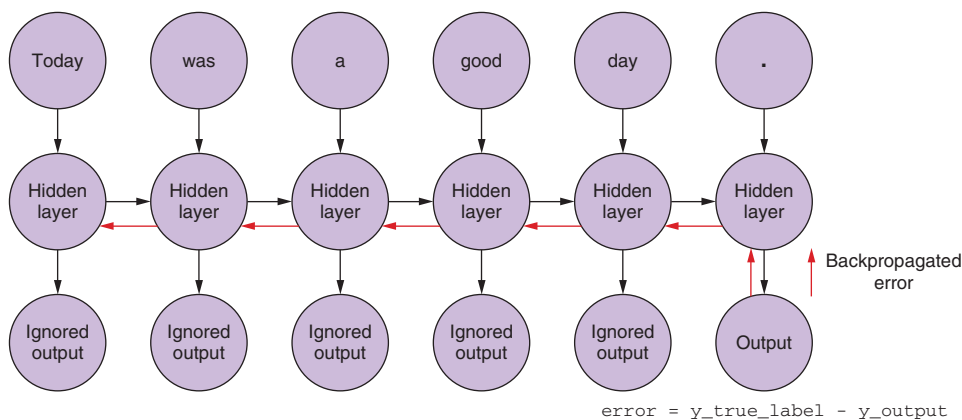


Figure 8.10 Backpropagation through time

TL;DR RECAP

- Break each data sample into tokens.
- Pass each token into a feedforward net.
- Pass the output of each time step to the input of the same layer alongside the input from the next time step.
- Collect the output of the last time step and compare it to the label.
- Backpropagate the error through the whole graph, all the way back to the first input at time step 0.

8.1.2 When do we update what?

You have converted your strange recurrent neural network into something that looks like a standard feedforward network, so updating the weights should be fairly straightforward. There's one catch though. The tricky part of the update process is the weights you're updating aren't a different branch of a neural network. Each leg is the *same* network at different time steps. The weights are the *same* for each time step (see figure 8.10).

The simple solution is that the weight corrections are calculated at each time step but not immediately updated. In a feedforward network, all the weight updates would be calculated once all the gradients have been calculated for that input. Here the same holds, but you have to hold the updates until you go all the way back in time, to time step 0 for that particular input sample.

The gradient calculations need to be based on the values that the weights had when they contributed that much to the error. Now here's the mind-bending part: a weight at time step t contributed something to the error when it was initially calculated. That *same* weight received a different input at time step $t+1$ and therefore contributed a different amount to the error then.

You can figure out the various changes to the weights (as if they were in a bubble) at each time step and then sum up the changes and apply the aggregated changes to each of the weights of the hidden layer as the last step of the learning phase.

TIP In all of these examples, you've been passing in a single training example for the *forward pass*, and then backpropagating the error. As with any neural network, this forward pass through your network can happen after each training sample, or you can do it in batches. And it turns out that batching has benefits other than speed. But for now, think of these processes in terms of just single data samples, single sentences, or documents.

That seems like quite a bit of magic. As you backpropagate through time, a single weight may be adjusted in one direction at one time step t (determined by how it reacted to the input at time step t) and then be adjusted in another direction at the time step for $t-1$ (because of how it reacted to the input at time step $t-1$), for a single data sample! But remember, neural networks in general work by minimizing a loss function, regardless of how complex the intermediate steps are. In aggregate, it will optimize across this complex function. As the weight update is applied once per data

sample, the network will settle (assuming it converges) on the weight for that input to that neuron that best handles this task.

BUT YOU DO CARE WHAT CAME OUT OF THE EARLIER STEPS

Sometimes you may care about the entire sequence generated by each of the intermediate time steps as well. In chapter 9, you'll see examples where the output at a given time step t is as important as the output at the final time step. Figure 8.11 shows a path for capturing the error at any given time step and carrying that backward to adjust all the weights of the network during backpropagation.

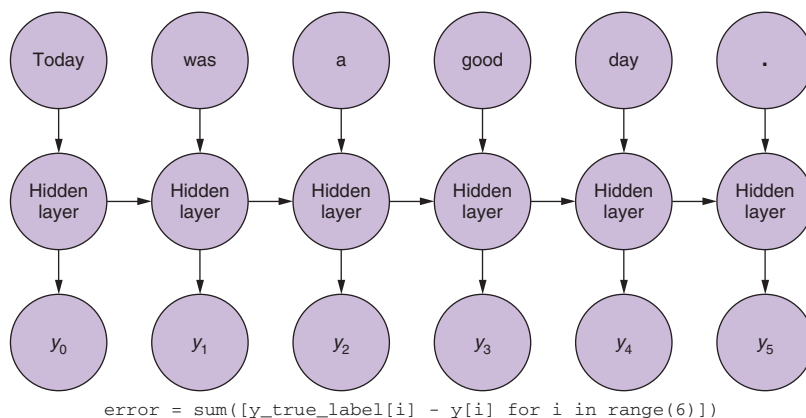


Figure 8.11 All outputs matter here

This process is like the normal backpropagation through time for n time steps. In this case, you're now backpropagating the error from multiple sources at the same time. But as in the first example, the weight corrections are additive. You backpropagate from the last time step all the way to the first, summing up what you'll change for each weight. Then you do the same with the error calculated at the second-to-last time step and sum up all the changes all the way back to $t=0$. You repeat this process until you get all the way back down to time step 0 and then backpropagate it as if it were the only one in the world. You then apply the grand total of the updates to the corresponding hidden layer weights all at once.

In figure 8.12, you can see that the error is backpropagated from each output all the way back to $t=0$, and aggregated, before finally applying changes to the weights. This is the most important takeaway of this section. As with a standard feedforward network, you update the weights *only after* you have calculated the proposed change in the weights for the entire backpropagation step for that input (or set of inputs). In the case of a recurrent neural net, this backpropagation includes the updates all the way back to time $t=0$.

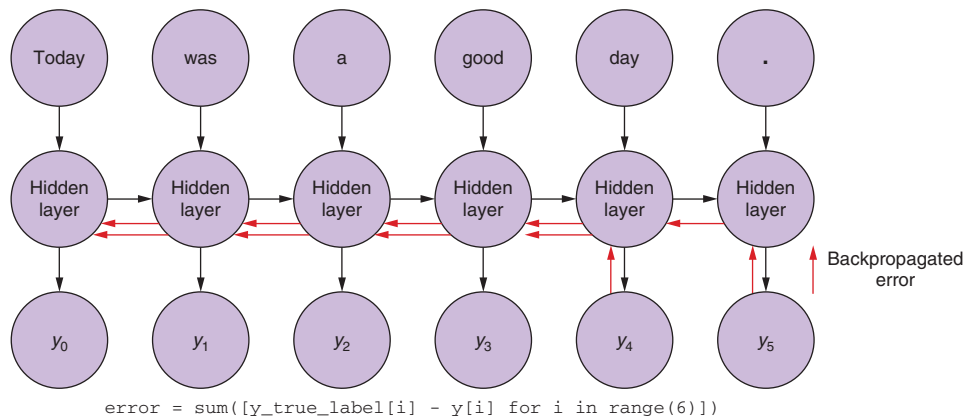


Figure 8.12 Multiple outputs and backpropagation through time

Updating the weights earlier would “pollute” the gradient calculations in the back-propagations earlier in time. Remember the gradient is calculated with respect to a particular weight. If you were to update it early, say at time step t , when you go to calculate the gradient at time step $t-1$, the weight’s value (remember it is the *same* weight position in the network) would’ve changed. Computing the gradient based on the input from time step $t-1$, the calculation would be off. You would be punishing (or rewarding) a weight for something it didn’t do!

8.1.3 Recap

Where do you stand now? You’ve segmented each data sample into tokens. Then one by one you fed them into a feed forward network. With each token, you input not only the token itself, but also the output from the previous time step. At time step 0, you input the initial token alongside 0, which ends up being a 0 vector, because there’s no previous output. You get your error from the difference between the output of the network from the final token and the expected label. You then backpropagate that error to the network weights, backward through time. You aggregate the proposed updates and apply them all at once to the network.

You now have a feedforward network that has some concept of time and a rudimentary tool for maintaining a memory of occurrences in that timeline.

8.1.4 There’s always a catch

Although a recurrent neural net may have relatively fewer weights (parameters) to learn, you can see from figure 8.12 how a recurrent net can quickly get expensive to train, especially for sequences of any significant length, say 10 tokens. The more tokens you have, the further back in time each error must be backpropagated. For each step back in time, there are ever more derivatives to calculate. Recurrent neural nets aren’t any less effective than others, but get ready to heat your house with your computer’s exhaust fan.